

PDF 1 of 4 — Original English Source

Availability

The complete, unedited lesson exactly as published — every paragraph, heading, numbered list, diagram, callout and example preserved. This document is the source of record for the topic.

100% Original Content

Nothing Summarized

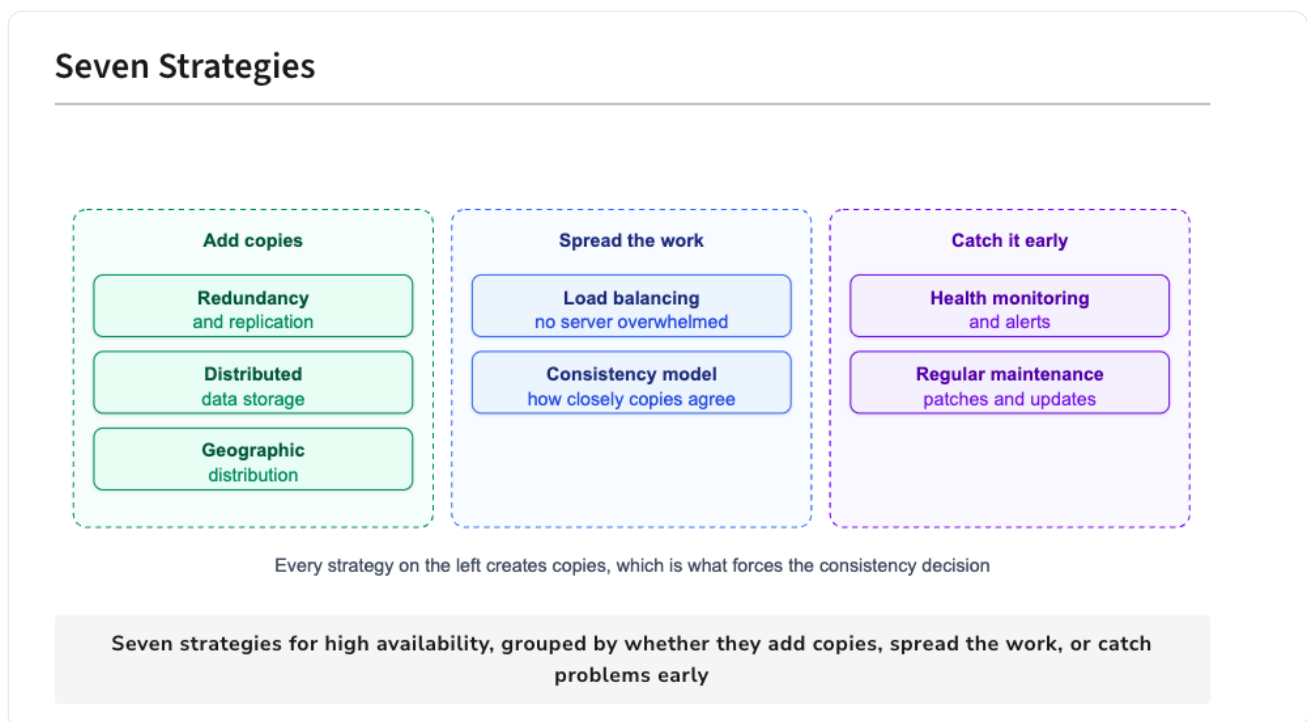
Nothing Removed

2. Availability is a measure of how accessible and reliable a system is to its users.

High availability is measured as uptime: the ratio of time the system is operational to the total time it is supposed to be operational. Getting that ratio close to one means minimizing both planned and unplanned downtime, removing single points of failure, and building in redundancy.

In a distributed system it means more than being switched on. It also means handling increased load and traffic without the performance collapsing, because a system that is technically running but too slow to use is not available in any way a user would recognize.

Seven Strategies



Seven strategies for high availability, grouped by whether they add copies, spread the work, or catch problems early

1. **Redundancy and replication.** Duplicate critical components or whole systems so that if one fails, the redundant copy takes over without interrupting service. Replication creates multiple copies of the data, so it stays available even when one copy is unreachable. This is what data centers do: several servers handle the workload, and when one crashes another absorbs it.
2. **Load balancing.** Distribute work across multiple servers so no single one is overwhelmed. This uses resources better, prevents bottlenecks, and matters most for web applications where many users arrive at once.
3. **Distributed data storage.** Store data across multiple locations or data centers. Replicating across geographically diverse sites means an outage at one site does not take the data with it.
4. **Consistency models.** How up to date every replica has to be is itself an availability decision, covered below.

5. **Health monitoring and alerts.** Watch performance, resource use and other metrics continuously, and trigger alerts when thresholds are crossed. The point is to find problems before users do.
6. **Regular maintenance and updates.** Patches, security fixes and routine checks reduce the chance of a failure that takes the system down. Availability is partly an operational habit, not only an architecture.
7. **Geographic distribution.** Deploy components across multiple locations or data centers, so that if one region or data center goes down, users can still reach the system from elsewhere. This is what protects against a localized incident taking out a global service.

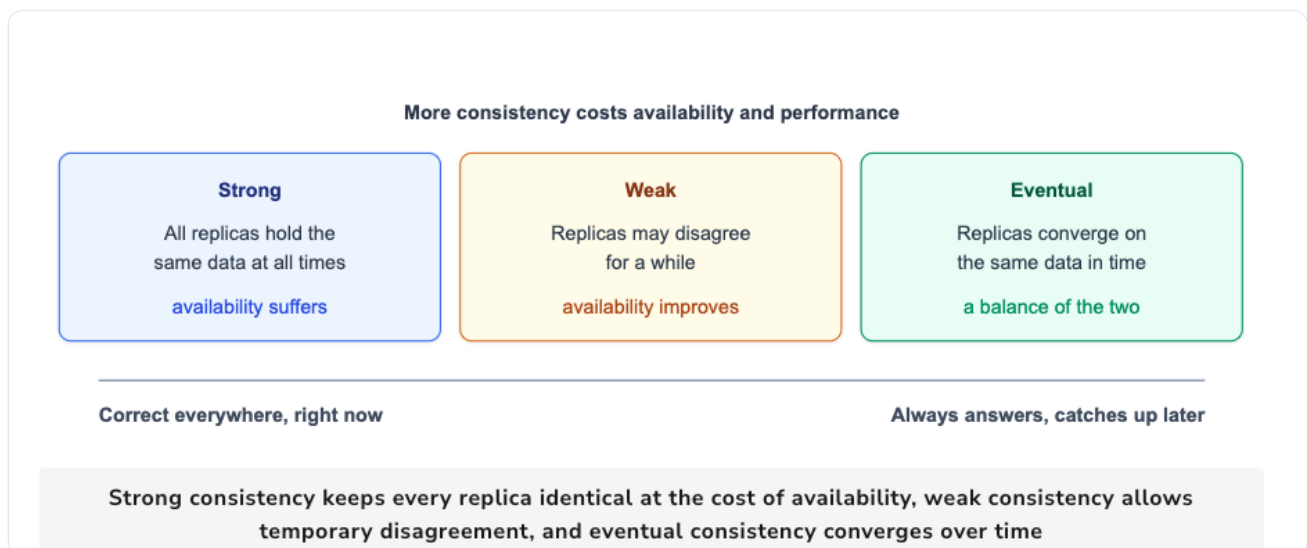
Consistency Costs Availability

A consistency model defines how a distributed system keeps a coherent, up to date view of its data across all replicas. The choice is a direct trade against availability and performance.

Strong consistency means all replicas hold the same data at all times. The cost is reduced availability and performance, because a write has to be settled everywhere before it counts.

Weak consistency allows replicas to disagree temporarily. The benefit is better availability and performance.

Eventual consistency guarantees that all replicas will converge on the same data eventually. It sits between the two, balancing consistency, availability and performance.



Strong consistency keeps every replica identical at the cost of availability, weak consistency allows temporary disagreement, and eventual consistency converges over time

This is the point where availability stops being a checklist and becomes a decision. Every strategy above adds copies of something, and the moment there is more than one copy, you have to say how closely they must agree.

Interview Tip

A number like "four nines" means nothing on its own in an interview. What earns credit is saying what is being counted. "Availability measured on the checkout API, not the whole site, because the marketing pages being slow is not an outage" shows you know availability is defined against a specific promise.

Key takeaway

Availability is how reachable and reliable a system is, measured as uptime against the time it should be operational. It is achieved through redundancy and replication, load balancing, distributed storage, health monitoring, regular maintenance and geographic distribution. Every one of those creates copies, which forces a consistency choice: strong consistency keeps replicas identical but costs availability, weak consistency allows temporary drift, and eventual consistency converges over time.

The next lesson, [Latency and Performance](#), looks at making the system fast rather than merely reachable.